Техническое предложение: Трёхконтурный автопилот для автономного наземного транспорта

Автор: Максим Тимошенко
Дата: 24.04.2026

Оглавление

1. Постановка задачи и исходные условия
2. Архитектура системы: общий обзор
    - 2.1. Три контура управления
    - 2.2. Принцип конкуренции (WTA)
    - 2.3. Структурная схема
3. Контур V_h (Витальный): детальная реализация
    - 3.1. Назначение
    - 3.2. Состав: сенсоры и данные
    - 3.3. Как работает: логика и циклы
    - 3.4. Код: вычисление приоритета и основные функции
    - 3.5. Таблица действий при активации
4. Контур V_e (Когнитивный): детальная реализация
    - 4.1. Назначение
    - 4.2. Состав: сенсоры и данные
    - 4.3. Как работает: нормальный режим и нештатные ситуации
    - 4.4. Код: вычисление приоритета и основной цикл
    - 4.5. Алгоритмы навигации
5. Контур V_s (Социальный): детальная реализация
    - 5.1. Назначение
    - 5.2. Состав: сенсоры и данные
    - 5.3. Как работает: оператор, спецтранспорт, регулировщик, коммуникация
    - 5.4. Код: вычисление приоритета и основные функции
6. Регуляторный предохранитель
7. Конкуренция контуров (арбитраж)
    - 7.1. Иерархия приоритетов
    - 7.2. Основной цикл арбитра (код)
    - 7.3. Диаграмма переходов
    - 7.4. Примеры конкурентных сценариев (таблицы)
8. Аппаратная платформа и схема соединений
9. Оценка стоимости и ожидаемый результат

1. Постановка задачи и исходные условия

1.1. Тип транспортного средства

- Легковой или грузовой автомобиль, оборудованный системами автоматического управления (drive-by-wire).
- Возможность вмешательства человека-оператора (водителя) в процесс управления.

1.2. Условия работы

Фактор	Описание
Спутниковая навигация	Может быть недоступна или подавлена (плотная городская застройка, тоннели).
Радиосвязь	Может отсутствовать. Система должна быть полностью автономна.
Метеоусловия	Дождь, снег, туман, гололёд, пыльные бури.
Дорожная среда	Городские улицы, трассы, бездорожье. Разметка, знаки, светофоры.
Участники движения	Другие автомобили (с водителями и без), пешеходы, велосипедисты, животные.
Бортовые сенсоры	Лидар, радары, камеры видимого и ИК-диапазона, IMU, колёсные одометры, CAN-шина автомобиля.

1.3. Проблема существующих решений

Существующие автопилоты, как правило, являются реактивными системами, основанными на жёстких правилах или обученных нейросетях. Они не имеют внутренней мотивации и не могут эффективно приоритизировать противоречивые задачи. Когда возникает конфликт между соблюдением ПДД и предотвращением столкновения, или между командой водителя и требованием безопасности, такие системы либо зависают, либо требуют ручного вмешательства.

Кроме того, существующие системы плохо справляются с нештатными ситуациями: отсутствующая разметка, неработающий светофор, жесты регулировщика. В этих случаях они, как правило, отключаются и передают управление водителю.

1.4. Цель разработки

Создать архитектуру бортовой системы управления, которая:

1. Является мотивационной, а не реактивной: у системы есть внутренние цели (безопасность, эффективность, социальная кооперация).
2. Разрешает конфликты целей через механизм динамической конкуренции, давая безусловный приоритет жизненно важным задачам.
3. Включает человека-оператора в контур управления как социальный сигнал высокой значимости.

4. Работает в условиях нарушенной дорожной инфраструктуры без отключения.
5. Осуществляет непрерывный мониторинг всех систем автомобиля.
6. Имеет стоимость дополнительного оборудования в пределах $100.

Отличия от существующих решений

Предлагаемая трёхконтурная архитектура принципиально отличается от всех известных автопилотов — как коммерческих (Tesla FSD, Waymo, Mobileye), так и экспериментальных. Ниже приведены ключевые отличия.

Параметр	Tesla FSD	Waymo	Mobileye	Предлагаемая система
Архитектура	Единая нейросеть (end-to-end)	Модульная, но без конкуренции	Правила + нейросеть (RSS)	Три контура с динамической конкуренцией (WTA)
Разрешение конфликтов	Неявное, через обучение	Жёсткая иерархия правил	Формальные правила (RSS)	Динамическое переключение через приоритеты
Приоритет безопасности	Не выделен	Зашит в правила	Зашит в RSS	Аппаратный перехват V_h за < 1 мс
Нештатные ситуации	Отключается, передаёт водителю	Отключается, передаёт оператору	Не заявлено	Адаптируется без отключения
Регулировщик	Не поддерживается	Не поддерживается	Не поддерживается	Распознаёт жесты, исполняет команды
Неработающий светофор	Не заявлено	Не заявлено	Не заявлено	Автоматически переключает логику перекрёстка
Отсутствие разметки	Частично, через дообучение	Требует HD-карты	Частично	Адаптируется: дорожный граф, край проезжей части, траектории других машин

Параметр	Tesla FSD	Waymo	Mobileye	Предлагаемая система
Водитель в контуре	Требует постоянного внимания	Отсутствует (L4)	Требует внимания	Равноправный участник с переменным приоритетом
Мониторинг систем авто	Базовый (ошибки)	Да	Да	Предиктивный, с упреждающими рекомендациями
Объяснимость решений	Низкая (чёрный ящик)	Средняя	Средняя	Высокая: всегда можно указать, какой контур победил и почему
Стоимость оборудования	$12,000+	$100,000+	$5,000+	< $100
Вычислительные требования	Мощный GPU на борту	Серверная стойка	Специализированный чип	MCU с NPU, энергопотребление < 5 Вт

Ключевое отличие: Все существующие системы являются реактивными — они реагируют на стимулы на основе обученных паттернов или жёстких правил. Предлагаемая система является мотивационной — каждый контур имеет собственную внутреннюю цель и конкурирует за управление, что позволяет разрешать конфликты без компромиссов.

---

2. Архитектура системы: общий обзор

2.1. Три контура управления автомобилем

Система состоит из трёх независимых логических контуров, работающих параллельно и конкурирующих за управление автомобилем по принципу **WTA** (Winner-Takes-All).

Контур	Назначение	Базовый приоритет	Максимальный приоритет
V_h (Витальный)	Безусловная безопасность. Предотвращение столкновений. Мониторинг критических систем автомобиля (двигатель, тормоза, топливо).	0	100 (безусловное прерывание)
V_e (Когнитивный)	Адаптивная навигация. Следование маршруту. Предсказание дорожной ситуации. Оптимизация скорости и манёвров.	40	80
V_s (Социальный)	Кооперация с другими участниками движения. Выполнение команд оператора (водителя). Взаимодействие со спецтранспортом и регулировщиком.	0	75

2.2. Принцип конкуренции

1. Каждый контур **непрерывно** (с частотой 100 Гц) вычисляет своё числовое значение приоритета P_i в диапазоне 0–100.
2. Управление получает контур с **максимальным** значением P_i.
3. Для предотвращения «дребезга» (хаотичного переключения) используется **гистерезис**: переключение на новый контур происходит только если его приоритет превышает приоритет текущего контура **на величину Δ = 15**.
4. **Исключение:** если P_vh превышает **критический порог** (значение 90), гистерезис игнорируется и управление передаётся V_h немедленно.

2.3. Структурная схема системы

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    СЕНСОРНЫЙ СЛОЙ (АППАРАТНАЯ ЧАСТЬ)                     │
├──────────────────────────────────────────────────────────────────────────┤
│  [Лидар] [Радар передний] [Радар задний] [Камеры x4-8] [GPS/ГЛОНАСС]     │
│  [IMU] [Ультразвуковые датчики x12] [Микрофоны x4] [V2X-модем]           │
│  [CAN-шина: двигатель, тормоза, руль, датчики состояния]                 │
└──────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                    ВЫЧИСЛИТЕЛЬНЫЙ СЛОЙ (ПРОГРАММНАЯ ЧАСТЬ)               │
```

```
┌──────────────┬──────────────┬──────────────────────────────┐
│  Контур V_h  │  Контур V_e  │          Контур V_s          │
│ (безопасность)│ (навигация)  │   (социум + водитель)        │
│  P_vh = ...  │  P_ve = ...  │         P_vs = ...           │
└──────┬───────┴──────┬───────┴──────────────┬───────────────┘
       │              │                      │
       └──────────────┴──────────────────────┘
                      │ (значения приоритетов)
                      ▼
          ┌──────────────────────┐
          │   АРБИТР (WTA logic) │
          │ Выбор победителя с   │
          │   гистерезисом       │
          └──────────┬───────────┘
                     │ (команда победившего контура)
                     ▼
          ┌──────────────────────┐
          │ РЕГУЛЯТОРНЫЙ         │
          │ ПРЕДОХРАНИТЕЛЬ       │
          │ (блокировка запрещённых│
          │  действий)           │
          └──────────┬───────────┘
                     │
                     ▼
┌──────────────────────────────────────────────────────────────┐
│                ИСПОЛНИТЕЛЬНЫЙ СЛОЙ (АППАРАТНАЯ ЧАСТЬ)        │
├──────────────────────────────────────────────────────────────┤
│ [Рулевое управление] [Тормозная система] [Силовая установка] │
│ [Световые приборы] [Звуковой сигнал] [Дисплей водителя]      │
└──────────────────────────────────────────────────────────────┘
```

---

3. Контур V_h (Витальный): детальная реализация для автомобиля

3.1. Назначение

Контур V_h — это контур безопасности, реализованный как аппаратно-программный блок с наивысшим приоритетом. Его задача — предотвращение столкновений и контроль критических параметров автомобиля.

Ключевое отличие от стандартных систем активной безопасности (AEB, ESP): стандартные системы работают как независимые подпрограммы, каждая из которых отвечает за свой узкий сценарий. Они не имеют общей картины и не могут разрешать конфликты между собой. Контур V_h агрегирует данные со всех сенсоров и датчиков

автомобиля в единую числовую оценку уровня угрозы. Когда эта оценка превышает порог, V_h получает полный контроль над исполнительными механизмами, временно отключая все остальные системы управления.

3.2. Состав: сенсоры и данные

Группа 1: Сенсоры окружения

Сенсор	Назначение	Данные
Передний радар (77 ГГц)	Обнаружение движущихся объектов	Расстояние, относительная скорость, азимут
Задний радар	Контроль транспорта сзади	Расстояние, относительная скорость
Передний лидар (ToF)	Обнаружение статических препятствий	Трёхмерное облако точек
Ультразвуковые датчики (12 шт.)	Контроль ближней зоны (0.1–5 м)	Расстояние до препятствия
Передняя стереокамера	Классификация объектов	Класс объекта (пешеход, авто, знак), расстояние
IMU (6-осевой)	Измерение ускорений и угловых скоростей	ax, ay, az, gx, gy, gz

Группа 2: Бортовая диагностика (состояние автомобиля)

Источник	Параметр	Единицы
CAN-шина (блок управления двигателем)	Температура охлаждающей жидкости	°C
CAN-шина	Давление масла	bar
CAN-шина	Обороты двигателя	RPM
CAN-шина	Уровень топлива / заряд батареи	%
CAN-шина (блок ABS/ESP)	Скорость каждого колеса	км/ч

Источник	Параметр	Единицы
CAN-шина (блок ABS/ESP)	Температура тормозных механизмов	°C
Датчики TPMS	Давление в каждой шине	bar
Блок управления (ECU)	Коды ошибок (DTC)	Список

3.3. Как работает: логика и циклы

Контур V_h работает циклически с частотой 100 Гц. Каждый цикл состоит из четырёх шагов:

Шаг 1: Сбор данных. Считываются показания всех сенсоров и CAN-шины. Данные проходят предобработку: фильтрация шумов, проверка на выход за границы диапазона, отбрасывание некорректных значений.

Шаг 2: Вычисление приоритета. На основе собранных данных вычисляется числовая оценка уровня угрозы — приоритет P_vh в диапазоне от 0 до 100. Чем выше число, тем серьёзнее угроза. Логика вычисления построена на суммировании вкладов от различных факторов:

- Время до столкновения (TTC) — главный фактор. При TTC < 0.5 секунды приоритет мгновенно достигает 100.
- Тип препятствия. Пешеход даёт больший вклад, чем неподвижный объект.
- Состояние систем автомобиля. Критический перегрев двигателя или падение давления масла дают вклад, сопоставимый с умеренной угрозой столкновения.
- Погодные условия. Аквапланирование или гололёд добавляют постоянный фоновый уровень тревоги.

Шаг 3: Сравнение с текущим активным контуром. Если P_vh превышает приоритет текущего контура управления с учётом гистерезиса ($\Delta = 15$), V_h перехватывает управление и выполняет предписанное защитное действие. Если P_vh достигает 90 или выше, гистерезис игнорируется и перехват происходит немедленно.

Шаг 4: Контроль исполнения. V_h удерживает управление до тех пор, пока его приоритет не опустится ниже приоритета другого контура. После этого управление возвращается тому контуру, который был активен ранее (как правило, V_e).

Как V_h сканирует угрозы и наращивает приоритет

Контур V_h не ждёт, пока случится авария. Он непрерывно, 100 раз в секунду, оценивает текущую обстановку и вычисляет число — приоритет P_vh. Это число не бинарное («опасно/безопасно»), а плавно нарастающее.

Вот как это происходит в реальной дорожной ситуации. Автомобиль едет по трассе со скоростью 90 км/ч. Впереди, в 80 метрах, другой автомобиль начинает резко тормозить.

Радар фиксирует, что дистанция сокращается, а относительная скорость растёт. V_h вычисляет время до столкновения: 80 метров / 25 м/с = 3.2 секунды.

При TTC = 3.2 секунды приоритет V_h составляет 30 единиц. Это выше, чем обычный фон (5–10), но недостаточно, чтобы перехватить управление у V_e, у которого приоритет 40. Поэтому V_h молчит, но уже «настороже».

Спустя секунду дистанция сокращается до 40 метров, TTC = 1.6 секунды. Приоритет V_h подскакивает до 60. Это уже достаточно, чтобы перехватить управление у V_e (60 > 40 + 15). V_h берёт управление на себя и начинает упреждающее торможение с усилием 50%.

Если водитель впереди продолжает тормозить, и TTC падает до 0.4 секунды, приоритет V_h достигает 100. Это безусловное прерывание. Гистерезис игнорируется. V_h даёт команду на экстренное торможение — тормозное усилие 100%, газ полностью сброшен. Автомобиль останавливается.

Как только TTC возвращается к безопасным значениям (> 4 секунд), приоритет V_h падает до 10. V_h возвращает управление тому контуру, который был активен ранее — как правило, V_e.

Как V_h следит за состоянием автомобиля

V_h — это не только про столкновения. Он также следит за «здоровьем» автомобиля точно так же, как водитель поглядывает на приборную панель. Но в отличие от водителя, V_h никогда не отвлекается.

Температура двигателя растёт. Обычно она держится около 90°C. Но вот она поднимается до 105°C — V_h добавляет 25 к приоритету. До 110°C — добавляет 50. До 120°C — добавляет 80. При 120°C приоритет V_h составляет 80, что гарантированно выше, чем у V_e (макс. 80, но с учётом гистерезиса V_h выигрывает, так как его приоритет превышает V_e + 15 при любом значении V_e ниже 65). V_h перехватывает управление и принудительно ограничивает мощность двигателя до 30%, одновременно выводя на дисплей водителя сообщение: «Перегрев двигателя. Мощность ограничена».

То же самое с давлением масла. Норма — 2–4 bar. При падении до 1.5 bar V_h добавляет 25. До 1.0 bar — добавляет 60. До 0.5 bar — добавляет 85. При 0.5 bar приоритет 85 — это почти критический порог. V_h выводит сообщение: «Давление масла критическое. Заглушите двигатель».

Предиктивный мониторинг

V_h отслеживает не только текущие значения, но и тренды. Если давление масла медленно падает на протяжении 1000 км, V_h предупредит об этом задолго до того, как оно достигнет критического уровня. Если тормозные колодки изнашиваются быстрее обычного, V_h порекомендует проверить их до того, как они полностью сотрутся.

3.4. Код: вычисление приоритета и основные функции

```
#define CRITICAL_THRESHOLD 90
```

```c
int compute_vh_priority(sensor_data_t *sensors) {
    int priority = 0;

    // БЛОК 1: УГРОЗА СТОЛКНОВЕНИЯ
    float ttc_front = calculate_ttc_front(sensors);

    if (ttc_front < 0.5f)       priority += 100;
    else if (ttc_front < 1.0f)  priority += 85;
    else if (ttc_front < 2.0f)  priority += 60;
    else if (ttc_front < 4.0f)  priority += 30;

    if (detect_pedestrian_in_path(sensors)) {
        float dist = get_pedestrian_distance(sensors);
        if (dist < 5.0f)       priority += 95;
        else if (dist < 15.0f) priority += 60;
        else if (dist < 30.0f) priority += 30;
    }

    // БЛОК 2: СИСТЕМЫ АВТОМОБИЛЯ
    if (sensors->fuel_level < 5.0f)       priority += 70;
    else if (sensors->fuel_level < 10.0f) priority += 40;

    if (sensors->engine_temp > 120.0f)    priority += 80;
    else if (sensors->engine_temp > 110.0f) priority += 50;

    if (sensors->oil_pressure < 0.5f)     priority += 85;
    else if (sensors->oil_pressure < 1.0f) priority += 60;

    float max_brake_temp = get_max_brake_temp(sensors);
    if (max_brake_temp > 500.0f)          priority += 70;
    else if (max_brake_temp > 400.0f)     priority += 40;

    // БЛОК 3: ДОРОЖНЫЕ УСЛОВИЯ
    if (detect_aquaplaning(sensors)) priority += 75;
    if (detect_black_ice(sensors))   priority += 30;

    if (priority > 100) priority = 100;
    return priority;
}

float calculate_ttc_front(sensor_data_t *sensors) {
    float min_ttc = 1000.0f;
```

```
    for (int i = 0; i < sensors->radar.num_targets; i++) {
        float d = sensors->radar.targets[i].distance;
        float v = sensors->radar.targets[i].relative_speed;
        if (v > 0.1f) {
            float ttc = d / v;
            if (ttc < min_ttc) min_ttc = ttc;
        }
    }

    float ego_speed = get_ego_speed(sensors);
    if (ego_speed > 0.5f) {
        float d_static = lidar_get_closest(sensors);
        if (d_static < 100.0f) {
            float ttc_static = d_static / ego_speed;
            if (ttc_static < min_ttc) min_ttc = ttc_static;
        }
    }

    return min_ttc;
}
```

3.5. Таблица действий при активации V_h

Триггер	P_vh	Действие	До какого момента
TTC < 0.5 сек	100	Экстренное торможение (тормоз 100%, газ 0%)	До TTC > 4.0 сек или полной остановки
TTC < 2.0 сек	60–85	Упреждающее торможение (тормоз 50%)	До TTC > 4.0 сек
Пешеход < 5 м	95	Экстренное торможение + звуковой сигнал	До остановки
Удар зафиксирован	100	Остановка, аварийка, отправка координат через V_s	До вмешательства водителя
Топливо < 5%	70	Построение маршрута к ближайшей АЗС (через V_e)	До заправки
Перегрев двигателя > 120°C	80	Ограничение мощности до 30%, уведомление водителя	До t < 105°C

Триггер	P_vh	Действие	До какого момента
Давление масла < 0.5 bar	85	Уведомление: «Заглушите двигатель»	До остановки и проверки
Перегрев тормозов > 500°C	70	Принудительное ограничение скорости 60 км/ч	До t < 350°C
Аквапланирование	75	Плавное торможение, руль прямо	До восстановления сцепления
Гололёд	30	Ограничение скорости 30 км/ч	До выезда на сухой участок

---

4. Контур V_e (Когнитивный): детальная реализация для автомобиля

4.1. Назначение

Контур V_e — это блок навигации и планирования. Он отвечает за то, чтобы автомобиль доехал из точки А в точку Б, соблюдая ПДД, оптимизируя время в пути и расход топлива. Это основной контур в нормальном режиме движения.

4.2. Состав: сенсоры и данные

Источник	Назначение	Данные
GPS/ГЛОНАСС-приёмник	Глобальное позиционирование	Широта, долгота, скорость, курс
IMU (6-осевой)	Инерциальная навигация при потере GPS	Ускорения, угловые скорости
Колёсные одометры	Измерение пройденного пути и скорости	Скорость каждого колеса, пробег

Источник	Назначение	Данные
HD-карта (microSD)	Дорожный граф, разметка, знаки, светофоры	Граф дорог, полосы, ограничения
Камеры кругового обзора (4–8 шт.)	Распознавание дорожной сцены	Разметка, знаки, светофоры, другие ТС
Данные о трафике (RDS/TMC или модем)	Загруженность дорог, ДТП, перекрытия	Скорость потока, аварии

4.3. Как работает

V_e работает в трёх режимах:

Режим 1: Нормальное движение. Автомобиль следует по маршруту, удерживая полосу, соблюдая скоростной режим и безопасную дистанцию. V_e выбирает оптимальную полосу, прогнозирует действия других участников движения, обрабатывает перекрёстки и светофоры.

Режим 2: Движение при нарушениях инфраструктуры. Если разметка отсутствует или не читается (снег, износ, вода), V_e не отключается, а переходит на позиционирование по дорожному графу из HD-карты, краю проезжей части и траекториям других автомобилей. Если светофор не работает, V_e переключает логику перекрёстка с регулируемого на нерегулируемый и ищет знаки приоритета.

Режим 3: Перестроение маршрута. По запросу от V_h (топливо на исходе), V_s (команда водителя) или при обнаружении затора V_e строит новый маршрут до цели или промежуточной точки.

Как V_e ведёт автомобиль в нормальном режиме

В нормальном режиме V_e — это основной контур. Он активен всегда, его базовый приоритет — 40. Водитель может вообще не вмешиваться в управление: V_e сам ведёт автомобиль по маршруту, удерживает полосу, соблюдает дистанцию и скоростной режим.

Каждые 100 миллисекунд V_e выполняет цикл:

1. Определяет своё точное положение на дороге с точностью до полосы.
2. Прогнозирует, куда поедут другие автомобили в ближайшие 10 секунд.
3. Выбирает оптимальную полосу — где быстрее поток, меньше препятствий и удобнее для предстоящего манёвра.
4. Рассчитывает оптимальную скорость — с учётом знаков, погоды, трафика и геометрии дороги.

5. Если впереди перекрёсток, V_e обрабатывает его: смотрит на светофор, знаки, другие автомобили и принимает решение — ехать, уступать или остановиться.

Как V_e адаптируется к нарушениям инфраструктуры

Ситуация: разметка на дороге стёрта или скрыта снегом. Обычный автопилот в этом случае отключается и передаёт управление водителю. V_e не отключается.

Вместо этого он переключается на альтернативные источники информации о положении на дороге:

- Дорожный граф из HD-карты. Даже если разметки не видно, V_e знает, сколько полос на этом участке и какова их ширина.
- Край проезжей части. Лидар позволяет определить границу между асфальтом и обочиной.
- Траектории других автомобилей. Если три машины впереди едут по параллельным траекториям, V_e может предположить, что это и есть полосы.

При этом V_e увеличивает боковые допуски и снижает уверенность в своих действиях. Это автоматически повышает фоновый приоритет V_h, который начинает более пристально следить за боковыми препятствиями.

Ситуация: светофор не работает. V_e видит выключенный светофор и мгновенно переключает логику перекрёстка с регулируемого на нерегулируемый. Он начинает искать знаки приоритета. Если знаков нет — применяет правило «помеха справа». При этом он продолжает следить за другими автомобилями: если кто-то явно намерен проехать первым, V_e уступит, даже если формально приоритет у него.

4.4. Код: вычисление приоритета и основной цикл

```
int compute_ve_priority(ve_state_t *state, sensor_data_t *sensors) {
    int priority = 40;

    state->route_error = calculate_cross_track_error(state, sensors);
    if (state->route_error > 500.0f)       priority += 25;
    else if (state->route_error > 200.0f)  priority += 15;
    else if (state->route_error > 50.0f)   priority += 10;

    float dist = get_distance_to_next_maneuver(state);
    if (dist < 200.0f)       priority += 20;
    else if (dist < 500.0f)  priority += 10;

    if (state->visibility < 100.0f)       priority += 20;
    else if (state->visibility < 300.0f)  priority += 10;
    if (state->friction_coefficient < 0.3f) priority += 15;

    if (get_traffic_flow_speed(state, sensors) < state->current_speed * 0.3f)
        priority += 15;
```

```
    if (priority > 80) priority = 80;
    return priority;
}

void execute_contour_ve(sensor_data_t *sensors) {
    localize_on_hd_map(&ve_state, sensors);
    predict_traffic(&ve_state, sensors);

    int target_lane = select_optimal_lane(&ve_state, sensors);
    if (target_lane != ve_state.current_lane)
        initiate_lane_change(&ve_state, target_lane);

    float target_speed = calculate_target_speed(&ve_state, sensors);
    adjust_speed(target_speed);
    adjust_steering(calculate_steering(&ve_state, sensors));

    if (approaching_intersection(&ve_state))
        handle_intersection(&ve_state, sensors);

    handle_traffic_lights(&ve_state, sensors);
    handle_traffic_signs(&ve_state, sensors);

    if (ve_state.reroute_requested) {
        replan_route(&ve_state, sensors);
        ve_state.reroute_requested = false;
    }
}
```

4.5. Алгоритмы навигации

4.5.1. Выбор оптимальной полосы

```
int select_optimal_lane(ve_state_t *state, sensor_data_t *sensors) {
    int num_lanes = get_num_lanes(state);
    float best_score = -1000.0f;
    int best_lane = state->current_lane;

    for (int lane = 0; lane < num_lanes; lane++) {
        float score = 0.0f;
        score += get_lane_flow_speed(lane, sensors) * 0.4f;
        score -= get_lane_obstacle_penalty(lane, sensors) * 0.3f;
        score += get_lane_turn_suitability(state, lane) * 0.2f;
        score += get_lane_surface_quality(state, lane) * 0.1f;
```

```
        if (score > best_score) { best_score = score; best_lane = lane; }
    }
    return best_lane;
}
```

4.5.2. Расчёт оптимальной скорости

```
float calculate_target_speed(ve_state_t *state, sensor_data_t *sensors) {
    float speed = get_speed_limit(state);
    if (state->visibility < 100.0f)       speed = min(speed, 40.0f);
    else if (state->visibility < 300.0f)  speed = min(speed, 70.0f);
    if (state->friction_coefficient < 0.3f) speed = min(speed, 30.0f);
    speed = min(speed, get_traffic_flow_speed(state, sensors) + 10.0f);
    speed = min(speed, get_max_speed_for_curve(state));
    speed = max(speed, get_min_road_speed(state));
    return speed;
}
```

4.5.3. Поведение при нарушениях инфраструктуры

Отсутствие разметки:

Когда разметка не видна, V_e переходит на позиционирование по дорожному графу из HD-карты, оценивает границы полосы по краю проезжей части через лидар и отслеживает траектории других автомобилей. Боковые допуски увеличиваются.

Неработающий светофор:

Когда светофор выключен или мигает жёлтым, V_e переключает логику перекрёстка с регулируемого на нерегулируемый и ищет знаки приоритета. Если знаков нет, применяется правило «помеха справа».

---

5. Контур V_s (Социальный): детальная реализация для автомобиля

5.1. Назначение

Контур V_s — это блок взаимодействия автомобиля с внешними агентами: водителем, другими участниками движения, спецтранспортом и дорожной инспекцией.

5.2. Состав: сенсоры и данные

Канал «Водитель — Автомобиль»:

Источник	Назначение	Данные
Микрофон (салон)	Голосовые команды водителя	Аудиопоток 16 кГц
Сенсорный экран / кнопки на руле	Явные команды	Нажатия, координаты
Камера в салоне	Жесты водителя	Изображение 640×480
Датчики руля и педалей	Ручное управление	Угол руля, положение педалей

Канал «Автомобиль — Внешний мир»:

Источник	Назначение	Данные
V2X-модем	Обмен с другими ТС и инфраструктурой	Пакеты: координаты, скорость, намерения
Внешние микрофоны (4 шт.)	Обнаружение сирен, окриков	Аудиопоток 16 кГц
Камеры кругового обзора	Распознавание жестов, спецтранспорта, регулировщика	Изображение

5.3. Как работает

V_s преобразует входные сигналы (голос, жесты, радиосообщения) в числовой приоритет. Если этот приоритет превышает приоритет текущего активного контура, V_s получает управление и исполняет команду.

Как V_s взаимодействует с водителем

Контур V_s воспринимает водителя не как «источник абсолютных команд», а как равноправного участника со своим приоритетом. Команды водителя не исполняются слепо — они проходят через арбитраж.

Водитель говорит: «Поехали быстрее». V_s преобразует это в приоритет 30. Сравниваем: V_e имеет приоритет 40. 30 < 40 + 15 (гистерезис). V_s не может перехватить управление. Но V_e «слышит» этот сигнал и, если это безопасно, может добровольно увеличить скорость в пределах разрешённого.

Водитель говорит: «Останови здесь». V_s преобразует это в приоритет 55. Сравниваем: 55 > 40 + 15 = 55. На грани, но V_s выигрывает и берёт управление. Но прежде чем остановиться, предохранитель проверяет: разрешена ли здесь остановка? Если это магистраль — остановка запрещена. Предохранитель блокирует действие. V_s сообщает водителю: «Здесь остановка запрещена. Ближайшее разрешённое место через 300 метров».

Водитель кричит: «Стой, опасно!». V_s переходит в режим EMERGENCY и выставляет приоритет 65. Это гарантированно выше, чем у V_e (макс. 80, но V_s с 65 > V_e с любым значением ниже 50). V_s перехватывает управление и выполняет экстренную остановку. Но даже здесь V_h продолжает следить: если остановка приведёт к столкновению с автомобилем сзади, V_h с приоритетом 100 перехватит управление и, возможно, вместо остановки выполнит манёвр уклонения с последующей остановкой.

Как V_s работает с регулировщиком

Обнаружив человека в форме с жезлом, V_s немедленно выставляет приоритет 70. Это выше, чем у V_e (макс. 80) при любом значении V_e ниже 55. Но это не безусловное значение.

V_s интерпретирует жест регулировщика и передаёт команду исполнительным механизмам. Например: «Регулировщик приказывает проезжать, хотя светофор красный». Автомобиль начинает движение.

Однако в этот же момент V_h обнаруживает пешехода, который решил перебежать дорогу. TTC с пешеходом — 0.3 секунды. V_h выставляет приоритет 100 — безусловное прерывание. Автомобиль экстренно тормозит, несмотря на приказ регулировщика.

Когда пешеход уходит, приоритет V_h падает, и V_s снова получает управление, продолжая выполнять команду регулировщика.

Градация приоритетов команд:

Тип команды	Приоритет	Пример
Фоновая информация	0–10	«Впереди затор»
Пожелание водителя	20–30	«Поехали быстрее»
Указание водителя	30–40	«Перестройся левее»
Требование водителя	55–65	«Останови здесь и сейчас»
Спецтранспорт с мигалками	50	«Уступи дорогу»
Команда регулировщика	70	«Проезжай на красный»
Сигнал тревоги водителя	65	«Стой, опасно!»

5.4. Код: вычисление приоритета и основные функции

```
int compute_vs_priority(vs_state_t *state, sensor_data_t *sensors) {
    int priority = 0;

    if (state->operator_mode == OPERATOR_EMERGENCY)      priority += 65;
    else if (state->operator_mode == OPERATOR_OVERRIDING) {
        if (is_stop_allowed_here()) priority += 55;
    }
    else if (state->operator_mode == OPERATOR_CORRECTING) priority += 30;
    else if (state->operator_mode == OPERATOR_NAVIGATING) priority += 25;

    if (state->emergency_vehicle_detected) {
        float dist = get_emergency_vehicle_distance(sensors);
        if (dist < 50.0f)       priority += 50;
        else if (dist < 200.0f) priority += 30;
    }

    if (detect_traffic_officer(sensors)) priority += 70;
    if (receive_v2x_lane_change_request(state)) priority += 20;

    if (priority > 75) priority = 75;
    return priority;
}
```

5.5. Невербальная коммуникация

Исходящий сигнал	Исполнение	Значение
Короткое мигание дальним	0.3 сек	«Пропускаю»
Двойное мигание аварийкой	1 сек	«Спасибо»
Указатель поворота заранее	За 5 сек до манёвра	«Планирую перестроение»
Короткий гудок	0.2 сек	«Осторожно»
Длинный гудок	1.0 сек	«Опасность»

---

6. Регуляторный предохранитель

Регуляторный предохранитель — это не контур, а набор безусловных ограничений, реализованных на аппаратном уровне. Он не участвует в конкуренции, а блокирует действия, нарушающие ПДД.

Список безусловных ограничений

Ограничение	Исключение
Проезд на красный	Команда регулировщика. Манёвр уклонения от столкновения
Выезд на встречную полосу дольше 3 сек	Объезд препятствия с разрешения V_h
Превышение скорости > 20 км/ч	Манёвр уклонения от столкновения
Остановка в запрещённом месте	Поломка автомобиля
Игнорирование знака STOP	Нет исключений
Задний ход по магистрали	Нет исключений

Код предохранителя

```
block_reason_t regulatory_check(sensor_data_t *sensors, action_t action) {

    if (action == ACTION_PROCEED_THROUGH_INTERSECTION) {
        if (get_traffic_light_state(sensors) == RED) {
            if (has_officer_override()) return BLOCK_NONE;
            if (is_critical_evasion())  return BLOCK_NONE;
            return BLOCK_RED_LIGHT;
        }
    }

    if (action == ACTION_ENTER_ONCOMING_LANE) {
        if (get_time_on_oncoming() > 3.0f && !is_obstacle_avoidance())
            return BLOCK_ONCOMING_LANE;
    }

    if (action == ACTION_SET_SPEED) {
        if (get_proposed_speed() > get_speed_limit() + 20.0f && !is_critical_evasion(
))
            return BLOCK_SPEEDING;
    }

    if (action == ACTION_STOP) {
```

```
        if (!is_stop_allowed_here() && !is_breakdown())
            return BLOCK_ILLEGAL_STOP;
    }

    return BLOCK_NONE;
}
```

---

7. Конкуренция контуров (арбитраж)

Как работает конкуренция контуров

Три контура — V_h, V_e, V_s — работают одновременно и непрерывно. Каждый из них вычисляет свой приоритет. Тот, у кого приоритет выше, получает управление.

Но переключение не происходит мгновенно при малейшем колебании приоритетов. Для этого используется гистерезис: новый контур должен быть «заметно увереннее» текущего — его приоритет должен превышать текущий минимум на 15 единиц.

Это предотвращает ситуацию, когда при равных приоритетах система начинает метаться между контурами туда-сюда несколько раз в секунду. Например, если V_e имеет приоритет 40, а V_s — 50, разница составляет 10. Этого недостаточно для переключения. V_e сохраняет управление. Только когда V_s поднимет свой приоритет до 55 или выше, он сможет перехватить управление.

Безусловное прерывание

Из этого правила есть одно исключение: критический порог V_h, равный 90. Если V_h достигает 90 или выше, гистерезис игнорируется. Управление передаётся V_h немедленно, на следующем же такте (через 10 миллисекунд).

Это реализует фундаментальный принцип: безопасность важнее всего. Никакие навигационные задачи, никакие команды водителя, никакие социальные сигналы не могут перевесить прямую угрозу жизни.

7.1. Иерархия приоритетов

Уровень	Контур	Диапазон	Может перехватить у кого
0 (высший)	**Регуляторный предохранитель**	∞ (абсолют)	Всех. Блокирует действие, не перехватывая управление
1	**V_h (Витальный)**	0–100	V_e, V_s

Уровень	Контур	Диапазон	Может перехватить у кого
2	**V_s (Социальный)**	0–75	V_e
3	**V_e (Когнитивный)**	40–80	Только удерживает себя

7.2. Основной цикл арбитра (100 Гц)

```
contour_t current_contour = CONTOUR_VE;
int current_priority = 40;

void arbitration_cycle(sensor_data_t *sensors) {

    if (detect_manual_override(sensors)) {
        execute_manual_mode(sensors);
        return;
    }

    int p_vh = compute_vh_priority(sensors);
    int p_ve = compute_ve_priority(&ve_state, sensors);
    int p_vs = compute_vs_priority(&vs_state, sensors);

    // Безусловное прерывание
    if (p_vh >= 90) {
        current_contour = CONTOUR_VH;
        current_priority = p_vh;
        execute_contour_vh(sensors);
        return;
    }

    // Выбор победителя
    contour_t winner;
    int winner_priority;

    if (p_vh >= p_ve && p_vh >= p_vs)      { winner = CONTOUR_VH; winner_priority = p
_vh; }
    else if (p_ve >= p_vh && p_ve >= p_vs) { winner = CONTOUR_VE; winner_priority = p
_ve; }
    else                                   { winner = CONTOUR_VS; winner_priority =
p_vs; }

    // Гистерезис
    if (winner != current_contour) {
        if (winner_priority > current_priority + 15) {
```

```
            current_contour = winner;
            current_priority = winner_priority;
        }
    } else {
        current_priority = winner_priority;
    }

    // Предохранитель
    action_t action = get_proposed_action(current_contour, sensors);
    if (regulatory_check(sensors, action) != BLOCK_NONE) {
        enforce_safe_action(sensors);
        return;
    }

    // Исполнение
    switch (current_contour) {
        case CONTOUR_VH: execute_contour_vh(sensors); break;
        case CONTOUR_VE: execute_contour_ve(sensors); break;
        case CONTOUR_VS: execute_contour_vs(sensors); break;
    }
}
```

7.3. Диаграмма переходов

```
┌──────────────────────────────────────────────────────────┐
│                    V_h (Витальный)                       │
│                    0 – 100                               │
│  P_vh >= 90 → Безусловное прерывание                     │
│  P_vh > P_ve + 15 → Выигрывает у V_e                     │
│  P_vh > P_vs + 15 → Выигрывает у V_s                     │
└──────────────────────────────────────────────────────────┘
        │                                  │
        ▼                                  ▼
┌──────────────────────────┐     ┌──────────────────────────┐
│    V_e (Когнитивный)     │◄────►│    V_s (Социальный)      │
│    40 – 80               │     │    0 – 75                │
│  Управляет в норме       │     │  Перехватывает при:      │
│  Уступает при:           │     │  P_vs > P_ve + 15        │
│  P_vs > P_ve + 15        │     │  (команда водителя,      │
│  P_vh > P_ve + 15        │     │   спецтранспорт,         │
│                          │     │   регулировщик)          │
└──────────────────────────┘     └──────────────────────────┘
```

7.4. Примеры конкурентных сценариев

Сценарий 1: Нормальное движение по трассе

Контур	Приоритет	Победитель
V_h	5	
V_e	**40**	**V_e**
V_s	0	

Сценарий 2: Внезапный пешеход в 5 метрах

Контур	Приоритет	Победитель
V_h	**95**	**V_h** (безусловно)
V_e	40	отстранён
V_s	0	отстранён

Сценарий 3: Регулировщик приказывает ехать на красный

Контур	Приоритет	Победитель
V_h	10	
V_e	40	
V_s	**70**	**V_s**

Сценарий 4: Регулировщик + встречный автомобиль летит наперерез

Контур	Приоритет	Победитель
V_h	**100**	**V_h** (безусловно)
V_e	40	отстранён
V_s	70	отстранён

Сценарий 5: Водитель хочет остановиться на магистрали

Контур	Приоритет	Результат
V_h	10	

Контур	Приоритет	Результат
V_e	40	
V_s	55	**Заблокировано предохранителем**

---

8. Аппаратная платформа и схема соединений

8.1. Выбор основного MCU

Предлагаемый вариант: WCH CH32V307

Параметр	Значение
Ядро	RISC-V RV32IMAF (с FPU)
Частота	до 144 МГц
Flash	256 КБ
RAM	64 КБ
Интерфейсы	SPI × 3, I2C × 2, UART × 4, SDIO, DCMI, CAN × 2
Цена	~$1.5 (опт)

Дополнительный MCU для V_h (опционально): WCH CH32V003 (48 МГц, $0.3).

8.2. Схема аппаратного перехвата управления

text

```
                    ┌──────────────────────┐
                    │      MCU V_h         │
                    │   WCH CH32V003 или   │
                    │   аналог (48 МГц, $0.3)  │
                    └──────────────────────┘
                              │
                              │
┌─────────────────────────────┼─────────────────────────────┐
│                             │                             │
▼                             ▼                             ▼
```

```
┌──────────────┐          ┌──────────────────────┐        ┌──────────────────────┐
│ Лидар (UART) │          │ МУЛЬТИПЛЕКСОР        │        │ ИСПОЛНИТЕЛЬНЫЕ       │
│ Радар (CAN)  │          │ 2:1 (ШИМ)            │        │ МЕХАНИЗМЫ            │
│ IMU (I2C)    │          │                      │        │                      │
│              │          │ Управляющий сигнал:  │        │ • Руль               │
│              │          │ OVERRIDE (GPIO)      │        │ • Тормоз             │
│              │          │                      │        │ • Газ                │
│              │          │ Вход A: от основного │        │                      │
│              │          │         MCU          │        │                      │
│              │          │ Вход B: от MCU V_h   │        │                      │
│              │          └──────────┬───────────┘        └──────────────────────┘
│              │                     │
│              │          ┌──────────┴───────────┐
│              │          │  ОСНОВНОЙ MCU        │
│              │          │  WCH CH32V307        │
│              │          │  (V_e + V_s + Арбитр)│
│              │          └──────────────────────┘
│              │
└──────────────┘
```

---

9. Примерная оценка стоимости и ожидаемый результат

9.1. Стоимость компонентов

Компонент	Модель	Цена (USD)
Основной MCU	WCH CH32V307	1.50
MCU V_h (опц.)	WCH CH32V003	0.30
IMU	ICM-42688-P	2.00
Барометр	BMP280	0.50
GPS/ГЛОНАСС	NEO-M8N	8.00
Камеры (x4)	OV7670	8.00
Лидар	TF-Luna	12.00
Радар	Аналог ARS408	30.00
V2X-модем	CC1101	3.00

Компонент	Модель	Цена (USD)
Микрофоны (x4)	MEMS	4.00
CAN-трансиверы	ISO1050 × 3	6.00
DC-DC преобразователь	12В → 5В/3.3В	2.50
Печатная плата, разъёмы, корпус	—	8.00
Итого		**~85.80**

9.2. Ожидаемый результат

Метрика	Без системы	С предлагаемой системой	Улучшение
Реакция на пешехода	~1.5 сек	< 10 мс	В 150 раз быстрее
Проезд нерегулируемых перекрёстков	Риск ДТП	Автоматически	Качественное
Движение без разметки	Ошибка или отключение	Адаптивно	Качественное
Реакция на регулировщика	Только человек	Автоматически	Качественное
Стоимость	$0 (человек-водитель)	< $100	Массово доступно

9.3. Заключение

Предлагаемая система позволяет за счёт программно-аппаратных средств стоимостью менее $100 создать трёхконтурный автопилот, который:

1. Не отключается при нарушениях дорожной инфраструктуры.
2. Понимает жесты регулировщика.
3. Взаимодействует с водителем как с равным участником, а не как с источником безусловных команд.
4. Следит за состоянием автомобиля и прогнозирует отказы.
5. Безопасен по дизайну — витальный контур V_h имеет аппаратный приоритет и может перехватить управление за < 1 мс.

Настоящий документ представляет собой эскизный проект и теоретическую проработку. Практическая реализация требует адаптации под конкретную модель автомобиля, калибровки коэффициентов и дорожных испытаний. Кроме того, описанная архитектура не является жёсткой или окончательной. Разработчики

вправе изменять, дополнять и адаптировать её под свои задачи и оборудование. Основная ценность документа — в принципе трёхконтурной конкуренции, а не в конкретных значениях порогов или выборе элементной базы.